

TinyML para Detecção de Doenças em Tomateiro: Três Arquiteturas de Inferência e Gap Laboratório-Campo

Namem Rachid Jaudy Neto¹

¹Instituto Federal de Educação, Ciência e Tecnologia de Mato Grosso – IFMT
Cuiabá – MT – Brasil

namem.rachid.jaudy@gmail.com

Abstract. *This paper presents Ceres Diagnóstico, a low-cost TinyML system for early detection of tomato leaf diseases integrating distributed inference across an embedded microcontroller, a mobile device, and a cloud API. A MobileNetV2 INT8 model (638 KB), trained on PlantVillage (88,949 augmented images) via two-phase transfer learning, is deployed across three inference architectures: ESP32-S3 edge (692 ms, offline), Android on-device via `tflite_flutter` (≈ 200 – 400 ms estimated, not empirically measured), and Django REST API (306 ms, cloud). Five training experiments document the impact of INT8 calibration (+33.8 pp over the uncalibrated model; 2.37 pp residual loss vs. float), synthetic background augmentation (ineffective), and real-data fine-tuning (+10 pp). The final model achieves 98.43% (float) on PlantVillage and 18–30% on three independent field datasets, quantifying the persistent lab-field gap.*

Keywords: *TinyML, MobileNetV2, ESP32-S3, plant disease detection, embedded machine learning, lab-field gap.*

Resumo. *Este artigo apresenta o Ceres Diagnóstico, sistema de baixo custo para detecção precoce de doenças foliares em tomateiro integrando TinyML, IoT e aplicativo móvel, com inferência distribuída entre microcontrolador embarcado, dispositivo móvel e nuvem. Um modelo MobileNetV2 INT8 (638 KB), treinado no PlantVillage (88.949 imagens com augmentation) via transfer learning em duas fases, é implantado em três arquiteturas de inferência: ESP32-S3 embarcado (692 ms, offline), on-device no Android via `tflite_flutter` (≈ 200 – 400 ms estimados, não medidos empiricamente) e API Django REST (306 ms, nuvem). Cinco experimentos documentam o impacto da calibração INT8 (+33,8 pp sobre o modelo não-calibrado; 2,37 pp de perda residual vs. float), augmentation sintética (ineficaz) e fine-tuning real (+10 pp). O modelo final atinge 98,43% (float) no PlantVillage e 18–30% em três datasets de campo, quantificando o gap laboratório-campo.*

Palavras-chave: *TinyML, MobileNetV2, ESP32-S3, detecção de doenças em plantas, aprendizado de máquina embarcado, gap laboratório-campo.*

1. Introdução

O tomateiro (*Solanum lycopersicum*) é uma das culturas de maior importância econômica no Brasil, com produção anual superior a 4 milhões de toneladas [FAO 2024]. Doenças

foliares como requeima (*Phytophthora infestans*), septoriose (*Septoria lycopersici*) e mancha-bacteriana (*Xanthomonas* spp.) podem causar perdas de até 100% da safra [EMBRAPA HORTALIÇAS 2023]. O diagnóstico tradicional depende de agrônomos especializados, inacessíveis à maioria dos pequenos produtores rurais brasileiros.

Estudos com visão computacional demonstraram acurácia elevada em condições controladas — Mohanty et al. [Mohanty et al. 2016] atingiram 99,35% no Plant-Village — mas quedas severas ao transferir modelos para o campo real: Singh et al. [Singh et al. 2020] documentaram $\approx 31\%$ sem adaptação de domínio e Xu et al. [Xu et al. 2024] relataram quedas de até 58 pp. Essa lacuna é aprofundada na Seção 3. Sistemas TinyML (*Tiny Machine Learning*) permitem inferência de redes neurais em microcontroladores sem dependência de nuvem [Warden and Situnayake 2019] — crítico para regiões com conectividade instável como o Centro-Oeste brasileiro.

Este trabalho apresenta o **Ceres Diagnóstico**, que: (1) cobre 10 classes de doenças; (2) demonstra, como prova de conceito, inferência no ESP32-S3 em 692 ms com modelo de 638 KB (sem câmera integrada neste ciclo); (3) implementa três arquiteturas de inferência complementares (ESP32-S3, Android on-device e API cloud) sobre o mesmo modelo; (4) valida quantitativamente o gap em 3 *datasets* independentes; (5) documenta cinco experimentos com resultados negativos reprodutíveis (*augmentation* sintética ineficaz) e positivos (*fine-tuning* real +10 pp). Código e modelos disponíveis em <https://github.com/Namem/extensao2>.

O desenvolvimento seguiu ciclos iterativos em contexto de projeto acadêmico de graduação sem financiamento externo dedicado. Restrições de prazo e a não-aquisição do módulo de câmera OV5640 no ciclo vigente impediram a conclusão do *pipeline* de captura ponta a ponta no ESP32-S3; em consequência, o caminho Mobile (②) tornou-se o principal veículo de validação funcional do produto neste ciclo, enquanto o caminho Edge (①) permanece como prova de conceito de inferência embarcada.

O restante deste artigo está organizado em seis seções, incluindo esta introdução. A Seção 2 apresenta o referencial teórico (TinyML, MobileNetV2, quantização INT8 e MQTT); a Seção 3 discute os trabalhos relacionados; a Seção 4 detalha a metodologia (arquitetura, treinamento, *dataset* e experimentos); a Seção 5 apresenta e discute os resultados; e a Seção 6 traz as considerações finais e os trabalhos futuros.

2. Referencial Teórico

2.1. TinyML e Inferência Embarcada

TinyML designa a execução de modelos de aprendizado de máquina em microcontroladores com restrições severas de memória (tipicamente <512 KB RAM) e energia (<1 W) [Warden and Situnayake 2019]. O TensorFlow Lite Micro (TFLite Micro) é o principal *framework* para esse paradigma, executando modelos quantizados diretamente no firmware sem sistema operacional. A ausência de dependência de rede torna o TinyML adequado para ambientes rurais com conectividade instável. A opção por um microcontrolador (ESP32-S3, $\approx$ R\$ 80) em vez de um computador de placa única (Raspberry Pi 4, $\approx$ R\$ 400+) deve-se ao menor custo, à ausência de sistema operacional (menor superfície de ataque e *boot* instantâneo) e ao consumo reduzido (<500 mW vs. >5 W), viabilizando o *deploy* rural de baixo custo.

2.2. MobileNetV2 e Quantização INT8

O MobileNetV2 [Howard et al. 2017, Sandler et al. 2018] introduziu os blocos *inverted residuals with linear bottleneck*, que combinam expansão de canais, convolução depthwise separável e projeção linear. Essa arquitetura reduz o número de parâmetros de 138M (VGG16) para 3,4M, e com fator de escala $\alpha = 0,35$ chega a apenas 0,5M parâmetros — compatível com a memória do ESP32-S3.

A quantização INT8 (inteiros de 8 bits) pós-treinamento [Jacob et al. 2018] converte pesos e ativações de FP32 (ponto flutuante de 32 bits) para inteiros via fatores de escala e zero-point por tensor. Sem um *representative dataset* de calibração, os fatores de escala são estimados de forma imprecisa, causando degradação severa. A Tabela 1 compara as principais arquiteturas de Redes Neurais Convolucionais (CNN) consideradas para implantação no ESP32-S3.

Tabela 1. Comparativo de arquiteturas CNN para implantação no ESP32-S3.

Modelo	Parâmetros	RAM	ESP32-S3?
VGG16	138M	>500 MB	Não
ResNet-50	25M	>100 MB	Não
EfficientNet-B0	5,3M	≈ 20 MB	Não
YOLO (mínimo)	≈ 6 M	>10 MB	Não*
MobileNetV1	4,2M	≈ 3 MB	Sim (inferior)
MobileNetV2	3,4M	<4 MB	Sim[†]

*YOLO: detector de objetos (bounding box), incompatível com classificação de folha.

[†]Para o caminho mobile (Android), modelos como EfficientNet-Lite0 e MobileNetV3-Small também seriam viáveis. A opção pelo mesmo MobileNetV2 INT8 foi adotada para permitir comparação direta entre os três caminhos de inferência sobre modelo idêntico, eliminando variáveis de arquitetura. Comparação empírica entre modelos no Android constitui trabalho futuro.

2.3. Protocolo MQTT e IoT Agrícola

O MQTT (*Message Queuing Telemetry Transport*) [OASIS Standard 2019] é um protocolo de mensageria projetado para dispositivos com recursos limitados e redes instáveis. Opera com cabeçalho mínimo de 2 bytes e suporta três níveis de qualidade de serviço (QoS 0, 1 e 2), tornando-o superior ao HTTP REST (*Representational State Transfer*; cabeçalho ≈ 800 bytes, sem QoS nativo) para comunicação ESP32 → servidor em ambientes rurais.

Estabelecidos esses fundamentos, a próxima seção posiciona o Ceres Diagnóstico frente aos trabalhos que aplicam tais técnicas à detecção de doenças em plantas.

3. Trabalhos Relacionados

Mohanty et al. [Mohanty et al. 2016] estabeleceram o estado da arte inicial com 99,35% de acurácia no PlantVillage, mas em condições controladas (fundo uniforme, iluminação padronizada). Singh et al. [Singh et al. 2020] introduziram o PlantDoc (2.569 imagens de campo real, 27 espécies) e documentaram queda para $\approx 31\%$ sem adaptação de domínio. Neste trabalho utiliza-se o subconjunto de tomateiro do PlantDoc: 677 imagens (*train*

split) para *fine-tuning* supervisionado no Exp. D e 676 imagens (*test split*) como conjunto de validação independente nunca visto durante treinamento (total: 1.353 imagens). A remoção manual do fundo das imagens PlantDoc elevou a acurácia de 29,73% para 70,53% (+40,8 pp) sem modificar o modelo [Singh et al. 2020], confirmando que redes treinadas no PlantVillage aprendem o fundo cinza como *feature* discriminativa, não as lesões.

Xu et al. [Xu et al. 2024] revisaram 42 trabalhos e quantificaram o gap em 29–58 pp, concluindo que a maioria dos modelos carece de validação em condições de campo. Barbedo [Barbedo 2019] documentou especificidade geográfica severa: modelos treinados em uma região apresentam queda significativa ao serem aplicados em variedades e condições climáticas distintas.

O Ceres Diagnóstico se diferencia por: (a) validação em 3 *datasets* de campo independentes em três continentes; (b) *pipeline* IoT (*Internet of Things*) com três arquiteturas de inferência (ESP32-S3, Android on-device, API cloud) em diferentes estágios de maturidade; (c) documentação de 5 experimentos com resultados negativos reproduzíveis [Pineau et al. 2021].

4. Metodologia

Trata-se de uma pesquisa aplicada de natureza experimental, organizada em ciclos iterativos de desenvolvimento e validação. O trabalho foi desenvolvido no Instituto Federal de Mato Grosso (IFMT), Cuiabá, sem financiamento externo dedicado. A Tabela 2 sumariza os recursos de *hardware* e *software* empregados.

Tabela 2. Recursos de hardware e software utilizados.

Etapa	Recurso	Versão / Especificação
Treinamento	GPU NVIDIA RTX 3060 Ti	8 GB VRAM, CUDA (WSL2 Ubuntu)
Treinamento	Python / TensorFlow	3.12 / 2.x
<i>Backend</i>	Django REST + PostgreSQL	Python 3.13 / PostgreSQL 18
Embarcado	ESP32-S3 N16R8	240 MHz, PSRAM 8 MB
<i>Firmware</i>	PlatformIO / ESP-IDF	5.x
Aplicativo	Flutter / <code>tf_lite_flutter</code>	0.12.1
Mensageria	MQTT (HiveMQ / Mosquitto)	TLS, porta 8883

O custo de *hardware* do nó embarcado é de aproximadamente R\$ 80 (ESP32-S3). O treinamento foi realizado utilizando uma GPU NVIDIA RTX 3060 Ti (8 GB VRAM) de uso próprio, e todo o *software* empregado é de código aberto ou gratuito, mantendo o custo total do projeto acessível ao pequeno produtor rural.

4.1. Arquitetura do Sistema

O sistema implementa **três caminhos de inferência complementares** sobre o mesmo modelo TFLite INT8 (638 KB), ilustrados na Figura 1:

(1) Caminho Edge (ESP32-S3) — *prova de conceito*: o modelo é executado diretamente no microcontrolador ESP32-S3 N16R8 (240 MHz, PSRAM 8 MB) via TensorFlow Lite Micro, sem qualquer dependência de rede. O resultado é publicado via MQTT sobre TLS ao *broker* HiveMQ Cloud. A imagem e as coordenadas GPS nunca

saem do dispositivo, garantindo privacidade por design. **Status de validação:** inferência CNN medida com 10 imagens pré-carregadas como arrays C (692 ms $\pm$ 1 ms); o *pipeline* de captura completo (câmera $\rightarrow$ inferência $\rightarrow$ MQTT) não foi concluído neste ciclo por restrição de prazo e pela não-aquisição do módulo OV5640.

(2) **Caminho Mobile on-device (Flutter + tflite_flutter)** — *principal veículo validado*: o mesmo modelo é carregado como *asset* no APK e executado localmente no Android via `tflite_flutter 0.12.1`, sem servidor. Latência estimada de 200–400 ms, funciona *offline*. O usuário alterna entre modo Local e Cloud diretamente na tela de câmera. **Status:** *pipeline* funcional câmera $\rightarrow$ app $\rightarrow$ modelo $\rightarrow$ resultado validado *end-to-end*; tornou-se o caminho primário de validação do produto diante das restrições que impediram a conclusão do caminho Edge.

(3) **Caminho Cloud (Flutter + Django)** — *validado end-to-end*: o aplicativo Flutter captura a imagem, envia via HTTPS à API Django (Railway), que executa o mesmo modelo TFLite via Python *subprocess* e retorna o diagnóstico em JSON. *Pipeline* completo câmera $\rightarrow$ app $\rightarrow$ API $\rightarrow$ modelo $\rightarrow$ resultado validado em produção.

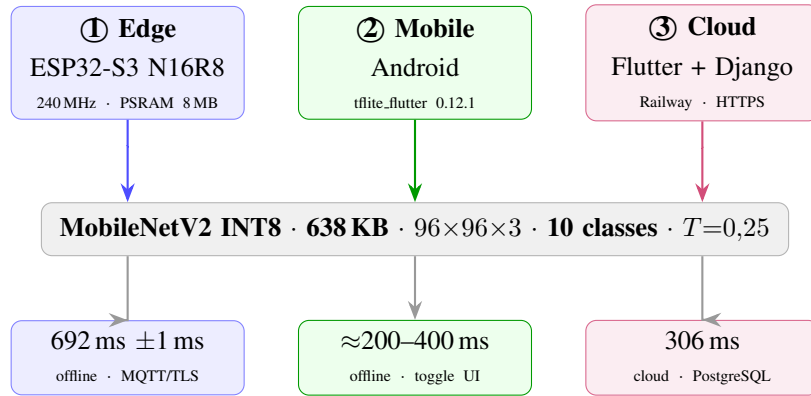


Figura 1. Três caminhos de inferência sobre o mesmo modelo MobileNetV2 INT8 (638 KB). ① Edge: TFLite Micro no ESP32-S3, imagens como arrays C (latência CNN pura, sem sensor). ② Mobile: `tflite_flutter` on-device, *offline* total. ③ Cloud: API Django (Railway), Python *subprocess* TFLite.

4.2. Treinamento do Modelo

Configuração: MobileNetV2 alpha= 0,35, entrada 96×96×3 RGB, backbone pré-treinado ImageNet, cabeça: GlobalAveragePooling + Dropout(0,3) + Dense(10, softmax).

Treinamento em duas fases: Fase 1 (10 épocas, LR= 1×10^{-3}) — backbone congelado, cabeça treinada; acurácia de validação estabiliza em 87,4%. Fase 2 (40 épocas, LR= 5×10^{-4}) — últimas 30 camadas descongeladas, EarlyStopping($p = 8$) + ReduceLROnPlateau($f = 0,5$). A melhor acurácia de validação (97,90%) ocorre na época 46, como ilustra a Figura 2.

Quantização INT8 calibrada: 50 batches do *val set* como *representative dataset* [Jacob et al. 2018]. Sem calibração, a quantização degrada a acurácia em −30,5 pp (Exp. A); com calibração, a perda residual cai para apenas 2,37 pp (98,13% float $\rightarrow$ 95,76% INT8 no *test set*). **Pós-processamento:** *temperature scaling* [Guo et al. 2017] divide os logits por um fator T antes do softmax, calibrando a confiança das predições sem alterar a classe predita. O valor $T=0,25$ foi determinado empiricamente no *val set*.

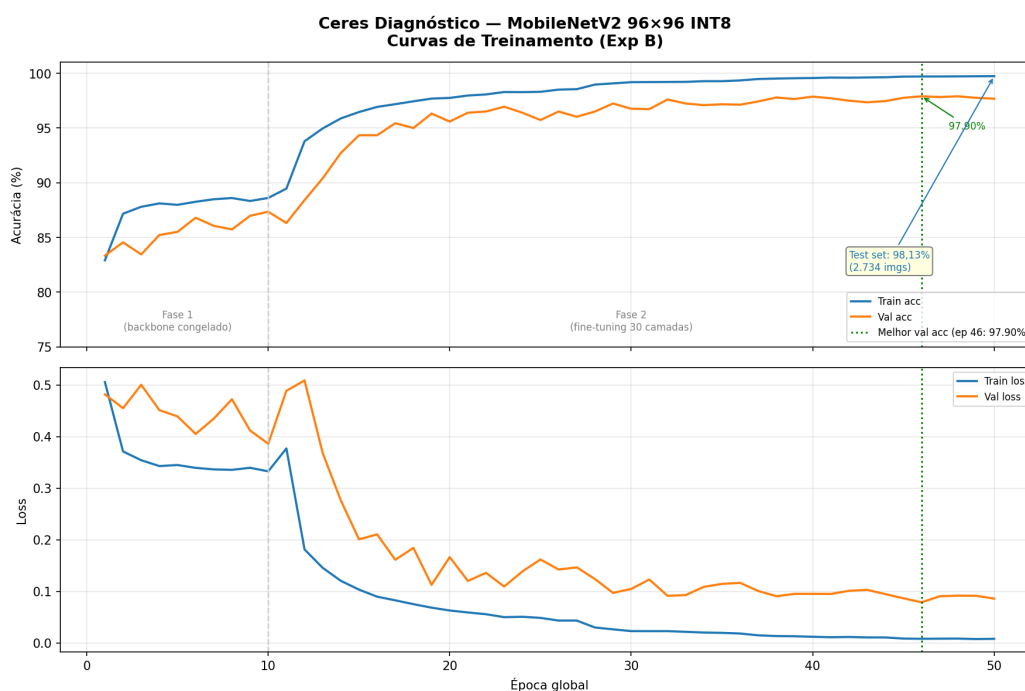


Figura 2. Curvas de treinamento do Exp. B (MobileNetV2 96×96). A linha vertical tracejada marca a transição da Fase 1 (*backbone* congelado) para a Fase 2 (*fine-tuning* das últimas 30 camadas). A melhor acurácia de validação (97,90%, época 46) corresponde a 98,13% no *test set* para o modelo float.

para maximizar a separação entre a classe correta e as demais; aplicado de forma idêntica nos três caminhos de inferência.

4.3. Dataset

PlantVillage [Hughes and Salathé 2015]: 18.160 imagens, 10 classes, CC BY 4.0. *Split* estratificado (seed= 42): 70% treino / 15% val / 15% teste. *Augmentation offline* (só treino): *flip* H/V, rotação $\pm 15^\circ$, brilho $\pm 20\%$. Resultado: 88.949 imagens de treino.

Datasets de campo real (validação independente): PlantDoc [Singh et al. 2020] — 1.353 imagens (EUA/Europa); Tomato-Village [Gehlot et al. 2023] — 217 imagens (Rajastão, Índia); Daffodil BD [Imtiaz et al. 2024] — 1.616 imagens (Bangladesh).

4.4. Experimentos

Cinco experimentos documentam a evolução do modelo (Tabela 3).

4.5. Firmware, Pipeline IoT e Integração Mobile

Firmware PlatformIO/ESP-IDF 5.x: modelo carregado como array C no flash, arena de inferência em PSRAM (200 KB de 512 KB alocados). A opção por imagens pré-carregadas como arrays C — em vez da câmera OV5640 — foi adotada por duas razões complementares: isolar a latência da CNN pura da latência de captura do sensor, garantindo medição comparável à literatura; e o fato de que a integração física da câmera OV5640 não foi concluída dentro do prazo disponível para este ciclo de desenvolvimento. A câmera constitui etapa futura de integração (item iii dos trabalhos futuros). O resultado da inferência CNN é publicado via MQTT com QoS 1 (*Quality of Service*) ao *broker* HiveMQ Cloud

Tabela 3. Configuração dos cinco experimentos de treinamento.

Exp.	Plataforma	Estratégia principal
A	Edge Impulse (nuvem)	Sem calibração INT8
B	TF local WSL2	2 fases + calibração INT8 (50 batches)
C	TF local WSL2	Exp. B + background aug. sintética (rembg U2-Net)
D	TF local WSL2	Exp. B + fine-tuning com Plant-Doc/train real
E	TF local WSL2	Exp. D + Focal Loss ($\gamma = 2$) + aug. agressiva

(porta 8883, TLS); Mosquito local utilizado apenas em desenvolvimento. O aplicativo Flutter captura a coordenada GPS (*Global Positioning System*) antes de cada inferência. O *cache* local é gerenciado pelo Drift (SQLite) com *SyncService*, que reenvia diagnósticos pendentes ao restaurar conectividade (*offline-first*).

A próxima seção apresenta os resultados obtidos com os cinco experimentos, a latência medida no ESP32-S3, o comparativo entre os três caminhos de inferência e a análise do gap laboratório-campo nos três *datasets* de campo.

5. Resultados e Discussão

5.1. Impacto da Calibração INT8

Tabela 4. Comparativo Exp. A (Edge Impulse) vs. Exp. B (TensorFlow local). O modelo efetivamente embarcado no ESP32-S3 é o INT8.

Métrica	Exp. A FP32	Exp. A INT8	Exp. B float	Exp. B INT8
Acurácia val set	92,5%	62,0%	97,90%	—
Acurácia test set	—	—	98,13%	95,76%
Macro F1	0,92	0,62	0,977	—
Tamanho modelo	1.637 KB	547 KB	—	638 KB

A ausência de *representative_dataset* no Exp. A causou queda de 30,5 pp (92,5% $\rightarrow$ 62,0%), consistente com Jacob et al. [Jacob et al. 2018]. Ressalta-se que Exp. A e Exp. B diferem não apenas na calibração INT8, mas também em plataforma (Edge Impulse vs. TensorFlow local), estratégia de treinamento (uma fase vs. duas fases) e hiperparâmetros; o efeito isolado da calibração não pode ser separado dessas demais variáveis. As classes mais afetadas foram D01_requeima (−30,7 pp) e D03b_mancha_alvo (−28,6 pp) — as com menos amostras no *val set*, evidenciando que a representatividade do *dataset* de calibração determina diretamente a qualidade dos fatores de escala por tensor.

O Exp. B, com calibração sobre 50 batches reais, reduz a perda de quantização a apenas 2,37 pp: o modelo float atinge 98,13% e o INT8 embarcado, 95,76% no *test set* — ainda assim +33,8 pp acima do INT8 não-calibrado do Exp. A (62,0%). A Figura 3 detalha os acertos e as confusões por classe do modelo INT8.

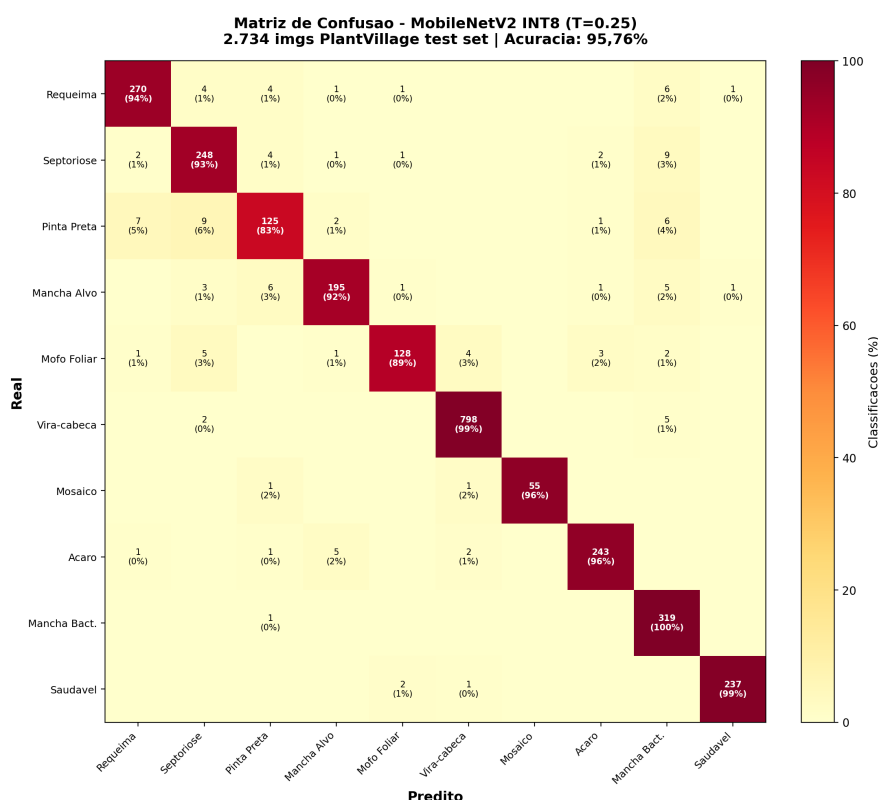


Figura 3. Matriz de confusão do modelo INT8 ($T = 0,25$) no *test set* do Plant-Village (2.734 imagens, acurácia 95,76%). As maiores confusões ocorrem entre pinta-preta, septoriose e requeima — doenças com lesões necróticas visualmente semelhantes.

5.2. Latência no ESP32-S3

A latência medida foi de **692 ms** (± 1 ms, determinístico), com 10/10 imagens corretas (imagens pré-carregadas como arrays C, sem câmera). O fator limitante é a ausência de unidade SIMD (*Single Instruction, Multiple Data*) INT8 dedicada no Xtensa LX7 — convoluções executadas via instruções escalares de propósito geral. No contexto agrícola, 692 ms representa apenas a etapa de inferência CNN; o tempo total de uso incluiria captura e posicionamento da folha, etapa não medida neste ciclo — tornando a latência da CNN provavelmente não o gargalo principal quando o *pipeline* de captura for concluído e validado empiricamente.

5.3. Gap Laboratório-Campo

Tomato-Village e Daffodil BD foram avaliados apenas nos modelos finais (Exp. D e E), como teste adicional de generalização geográfica; os experimentos B e C, anteriores, não foram reexecutados sobre eles — daí as células “—” na Tabela 5.

Background augmentation sintética (Exp. C): remoção de fundo com U2-Net [Qin et al. 2020] e recomposição sobre fundos do PlantDoc não melhorou o desempenho de campo (+0 pp vs. Exp. B). Mesmo após 177.698 composições sintéticas, o modelo nunca prediz folha saudável em campo (0,0%), indicando que o domínio sintético não captura a distribuição real. Resultado negativo documentado segundo as diretrizes de Pineau et al. [Pineau et al. 2021].

Tabela 5. Acurácia por experimento em lab e campo (três datasets independentes).

Exp.	PlantVillage (lab)	PlantDoc (campo)	Tomato-Village (campo)	Daffodil BD (campo)
B – baseline	98,13%	20,77%	—	—
C – aug. sintét.	96,20%	20,24%	—	—
D – fine-tun. real	97,55%	30,43%	11,52%	—
E – Focal Loss	98,43%	30,43%	27,65%	18,13%

Coluna lab: acurácia do modelo float no *test set*; o INT8 embarcado do Exp. B atinge 95,76% (Fig. 3).

Colunas de campo: modelo INT8 (mesma quantização do ESP32-S3).

Fine-tuning com dados reais (Exp. D): 677 imagens do PlantDoc/train (*oversampling* 10×, sem dados adicionais) produziram ganho de +10 pp (20,77% → 30,43%) avaliado no PlantDoc/test (676 imagens nunca usadas em treinamento). O fator limitante é o **volume de dados de campo**, não o método.

Focal Loss (Exp. E): Focal Loss ($\gamma = 2$) [Lin et al. 2017] melhorou consistência entre *datasets* (+16 pp no Tomato-Village: 11,52% → 27,65%).

Gap geográfico: EUA/Europa (30,43%) > Índia (27,65%) > Bangladesh (18,13%), confirmando Barbedo [Barbedo 2019]: especificidade geográfica em função de variedades locais e condições tropicais.

Colapso de classe: na análise da distribuição de predições do Exp. D sobre o Tomato-Village, 73% das folhas da classe saudável foram erroneamente classificadas como D02_septoriose — padrão típico de colapso sob *shift* de domínio extremo, parcialmente mitigado no Exp. E com Focal Loss ($\gamma = 2$), que penaliza predições excessivamente confiantes em classes dominantes.

5.4. Comparativo Edge, Mobile e Cloud

Tabela 6. Comparativo dos três caminhos de inferência.

Aspecto	Edge – ESP32-S3	Mobile – Android	Cloud – Django
Latência inferência	692 ms (± 1 ms)	$\approx 200\text{--}400$ ms*	306 ms (subprocess)
Latência end-to-end	692 ms	$\approx 200\text{--}400$ ms*	2.333 ms (HTTP dev)
Funciona offline	Sim	Sim	Não
Privacidade (LGPD)	Total	Total	Imagem transmitida
Custo adicional	$\approx$ R\$ 80	Zero (app)	Servidor PC/cloud
Status validação	Parcial**	End-to-end	End-to-end

*Estimada com base em benchmarks públicos de `tf.lite_flutter` 0.12.1 em dispositivos *mid-range*; não medida empiricamente neste ciclo por restrição de tempo — constitui trabalho futuro (item iv).

**Inferência CNN validada (692 ms, 10 imagens); *pipeline* câmera → inferência → MQTT não concluído por restrição de prazo e não-aquisição do módulo OV5640 neste ciclo.

Quando o pipeline de captura do ESP32-S3 for concluído (câmera OV5640 integrada), o caminho Edge será superior para zonas rurais sem conectividade e cenários

de privacidade total. No ciclo atual, o caminho Mobile é o único com validação end-to-end funcional offline. A latência Cloud de 2.333 ms reflete servidor de desenvolvimento (*single-thread*); estima-se <200 ms em produção com Gunicorn *multi-worker*.

6. Considerações Finais

O Ceres Diagnóstico demonstrou a viabilidade técnica de uma arquitetura TinyML distribuída — MobileNetV2 INT8 (638 KB) validado no ESP32-S3 (692 ms, prova de conceito) e em produção via mobile e cloud (end-to-end). Contudo, a acurácia de campo de 18–30% em três *datasets* independentes evidencia que o sistema não está pronto para implantação produtiva: dados de campo representativos e *domain adaptation* constituem pré-requisitos para uso agrícola real. As principais contribuições são:

1. **Pipeline reproduzível:** PlantVillage → 88.949 imgs → MobileNetV2 INT8 → ESP32-S3, código aberto (<https://github.com/Namem/extensao2>).
2. **Quantificação da calibração INT8:** +33,8 pp com *representative_dataset* (62,0% → 95,76%) vs. quantização automática, com perda residual de apenas 2,37 pp frente ao modelo float.
3. **Benchmark em 3 *datasets* de campo:** documentação do gap lab-campo (98,43% → 18–30%) e sua natureza geográfica.
4. **Resultado negativo reprodutível:** *augmentation* sintética ineficaz; *fine-tuning* com dados reais confirma volume como fator limitante.
5. **Três arquiteturas de inferência comparadas:** ESP32-S3, Android on-device e API cloud sobre o mesmo modelo INT8.

Limitações: A latência de 692 ms ficou acima da meta de 300 ms (Xtensa LX7 sem acelerador SIMD INT8). A câmera OV5640 não foi integrada neste ciclo — a validação usou imagens embarcadas como arrays C. O gap laboratório-campo (98,43% → 18–30%) persiste como problema aberto.

O projeto foi concebido originalmente com o ESP32-S3 como produto primário. Ao longo do desenvolvimento, duas restrições impediram a conclusão do *pipeline* embarcado ponta a ponta: (a) **prazo** — o semestre letivo não comportou a integração completa do firmware de captura; (b) **recurso de hardware** — o módulo de câmera OV5640 não foi adquirido e integrado no período disponível. Como resultado, o caminho Edge permaneceu como prova de conceito (inferência CNN validada, latência medida), e o caminho Mobile (Android + `tf_lite_flutter`) passou a ser o principal veículo de validação funcional do produto. Essa reorientação de escopo é documentada de forma transparente para que os resultados sejam interpretados corretamente: as contribuições de *pipeline* reproduzível, calibração INT8 e gap laboratório-campo são válidas e independentes do caminho de inferência; a validação de produto completo com câmera embarcada constitui etapa futura. Adicionalmente, a comparação entre modelos no caminho mobile limitou-se ao MobileNetV2 por restrição de tempo; alternativas como MobileNetV3-Small e EfficientNet-Lite0 não foram avaliadas.

Lições aprendidas: a principal lição deste ciclo é que, no domínio agrícola, o volume e a representatividade dos dados de campo pesam mais que a sofisticação do método de treinamento — o *fine-tuning* com apenas 677 imagens reais (Exp. D) superou a *augmentation* sintética em larga escala (Exp. C, 177.698 composições). O resultado negativo do Exp. C, documentado de forma reprodutível, evidencia que recompor folhas sobre

fundos naturais não substitui a aquisição de dados reais. A calibração INT8 com *representative dataset*, por sua vez, mostrou-se determinante: um detalhe de implementação (−30,5 pp se omitido) com impacto maior que mudanças de arquitetura.

6.1. Trabalhos Futuros

Os próximos passos incluem: (i) coleta de *dataset* em lavouras de Sorriso-MT para *fine-tuning* supervisionado com variedades locais; (ii) *domain adaptation* adversarial (DANN [Ganin et al. 2016]) sem necessidade de rótulos de campo; (iii) integração da câmera OV5640 para validação do *pipeline* de captura completo; (iv) medição empírica da latência de inferência on-device no Android via `tflite_flutter`; (v) validação de usabilidade com produtores rurais de Mato Grosso.

Uso de Inteligência Artificial Generativa

Declara-se que ferramentas de inteligência artificial generativa (Claude, Anthropic) foram utilizadas como auxílio na revisão gramatical, reorganização de parágrafos e sugestão de formatação LaTeX. Nenhum conteúdo técnico — experimentos, dados, análises, interpretações ou conclusões — foi gerado por IA; todo o desenvolvimento, execução e análise são de autoria e responsabilidade exclusivas do autor.

Referências

- Barbedo, J. G. A. (2019). Plant disease identification from individual lesions and spots using deep learning. *Biosystems Engineering*, 180:96–107.
- EMBRAPA HORTALIÇAS (2023). Doenças do tomateiro. Disponível em: <https://www.embrapa.br/hortalicas/tomate/doencas>. Acesso em: abr. 2026.
- FAO (2024). FAOSTAT — production: Crops and livestock products. Disponível em: <https://www.fao.org/faostat/en/#data/QCL>. Acesso em: jun. 2026.
- Ganin, Y., Ustunova, E., Ajakan, H., Germain, P., Larochelle, H., Laviolette, F., Marchand, M., and Lempitsky, V. (2016). Domain-adversarial training of neural networks. *Journal of Machine Learning Research*, 17(59):1–35.
- Gehlot, M., Saxena, R. K., and Gandhi, G. C. (2023). Tomato-Village: A dataset for end-to-end tomato disease detection in a real-world environment. *Multimedia Systems*, 29:3305–3328.
- Guo, C., Pleiss, G., Sun, Y., and Weinberger, K. Q. (2017). On calibration of modern neural networks. In *International Conference on Machine Learning (ICML)*, pages 1321–1330.
- Howard, A. G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., Andreetto, M., and Adam, H. (2017). MobileNets: Efficient convolutional neural networks for mobile vision applications.
- Hughes, D. P. and Salathé, M. (2015). An open access repository of images on plant health to enable the development of mobile disease diagnostics through machine learning and crowdsourcing.
- Imtiaz, A., Swapnil, F. B. I., Masud, S. R., and Karmaker, D. (2024). Tomato leaf diseases dataset. Mendeley Data, v1. <https://data.mendeley.com/datasets/>

93h9p62kg4/1. Daffodil International University, Khagan/Charabag, Bangladesh.
Artigo: <https://doi.org/10.1016/j.dib.2025.111520>.

- Jacob, B., Kligys, S., Chen, B., Zhu, M., Tang, M., Howard, A. G., Adam, H., and Kalenichenko, D. (2018). Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2704–2713.
- Lin, T.-Y., Goyal, P., Girshick, R., He, K., and Dollár, P. (2017). Focal loss for dense object detection. In *IEEE International Conference on Computer Vision (ICCV)*, pages 2999–3007.
- Mohanty, S. P., Hughes, D. P., and Salathé, M. (2016). Using deep learning for image-based plant disease detection. *Frontiers in Plant Science*, 7:1419.
- OASIS Standard (2019). MQTT version 5.0. Disponível em: <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>. Acesso em: abr. 2026.
- Pineau, J., Vincent-Lamarre, P., Sinha, K., Larivière, V., Beygelzimer, A., d’Alché Buc, F., Fox, E., and Larochelle, H. (2021). Improving reproducibility in machine learning research. *Journal of Machine Learning Research*, 22(164):1–20.
- Qin, X., Zhang, Z., Huang, C., Dehghan, M., Zaiane, O. R., and Jagersand, M. (2020). U2-Net: Going deeper with nested U-structure for salient object detection. *Pattern Recognition*, 106:107404.
- Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., and Chen, L.-C. (2018). MobileNetV2: Inverted residuals and linear bottlenecks. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4510–4520.
- Singh, D., Jain, N., Jain, P., Kayal, P., Kumawat, S., and Batra, N. (2020). PlantDoc: A dataset for visual plant disease detection. In *Proceedings of the 7th ACM IKDD CoDS and 25th COMAD*, pages 249–253.
- Warden, P. and Situnayake, D. (2019). *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. O’Reilly Media.
- Xu, M., Park, J.-E., Lee, J., Yang, J., and Yoon, S. (2024). Plant disease recognition datasets in the age of deep learning: challenges and opportunities. *Frontiers in Plant Science*, 15.